

FINAL PROJECT REPORT

Cyber Security Internship

Zaalima Development Pvt. Ltd.

Internship duration	Role
May 2026 - August 2026	Cyber Security Intern

Prepared by: Vineet Khatri

Submission date: 20-08-2026

Certificate / Internship Information

This report documents the work undertaken during a three-month Cyber Security Internship at Zaalima Development Pvt. Ltd. It is prepared from the submitted project repository and the organization's project-plan document. Fields that were not supplied are intentionally left for completion.

Field	Details
Intern name(s)	Vineet Khatri; Kashyap Shah; Sravani Bandlamoodi
Organization	Zaalima Development Pvt. Ltd.
Role	Cyber Security Intern
Duration	May 2026 - August 2026 (three months)

Acknowledgement

We sincerely thank Zaalima Development Pvt. Ltd. for the opportunity to work on cybersecurity-focused prototypes during this internship. The assigned project plans provided clear problem contexts spanning AI security, identity threat detection, and incident response. We also acknowledge the guidance, review expectations, and emphasis on disciplined Git-based development that shaped the work documented in this report.

Executive Summary

This internship produced three local/prototype cybersecurity implementations: an Enterprise LLM & GenAI Security Gateway, an Identity Threat Detection and Response (ITDR) Platform, and an Automated Ransomware Containment & Incident Response Orchestrator. The repository demonstrates modular Python implementations, FastAPI interfaces, local datasets and stores, reporting artifacts, and security-focused workflows. The projects are evaluated here against their actual evidence rather than against the broader enterprise target architecture alone.

The first prototype applies prompt inspection, regex and Microsoft Presidio PII analysis, prompt-injection checks, audit logging, local SQLite persistence, and a mock LLM response path. The ITDR prototype processes a local identity-event CSV through rule-based detections and risk scoring, writes alerts/timeline artifacts locally, and exposes dashboard-oriented API endpoints. The ransomware prototype coordinates local alert data through containment, user/session, forensic, tracking, ticketing, notification, summary, and metrics modules. Several of the planned external integrations remain future scope.

Internship Overview

The official plan assigned four-week development targets for each project and required repository creation, daily commits, professional commit messages, branch management, and collaborative workflows. The submitted top-level README identifies the internship period as May 2026 to August 2026 and lists the three project areas. This report distinguishes the official intended architecture from the evidence in the extracted source tree.

Project	Prototype focus	Primary repository evidence
1	LLM prompt security gateway	FastAPI, Presidio dependency, SQLite, audit logs, Dockerfile, screenshots
2	Identity threat detection and response	CSV event data, Pandas/Scikit-learn dependencies, SQLite, FastAPI, detection modules
3	Ransomware response orchestration	CSV alerts, Python playbooks, JSON/log tracking, FastAPI endpoints

Project 1 - Enterprise LLM & GenAI Security Gateway

Objective and Problem Statement

The official objective was a secure proxy between internal users/applications and external LLM APIs, intended to reduce sensitive-data leakage and prompt-injection risk. The implemented prototype focuses on inspecting user prompts, redacting recognized sensitive patterns, blocking selected malicious instruction patterns, and creating audit records before returning a sanitized result or mock response.

Official Requirements and Planned Architecture

The plan specified Python and FastAPI, with LangChain, Microsoft Presidio, Rebuff, PostgreSQL, Redis, Docker, and Kubernetes named as target technologies. It proposed authentication and rate limiting in week 1; PII/PHI redaction and unit tests in week 2; prompt-injection and response-safety controls in week 3; then dashboarding, RBAC, containerization, Kubernetes scaling, and documentation in week 4. These are requirements and targets, not automatic evidence of implementation.

Actual Implementation

The implementation centers on `src/api.py` and `src/security_gateway.py`. The `/analyze` endpoint requires a static X-API-Key value, increments in-memory dashboard counters, performs regex-based and Presidio analysis, checks two keyword-driven prompt-injection paths, sanitizes the prompt, and saves a text and SQLite audit record for non-blocked requests. The code recognizes email, 10-digit phone number, card-like values, PAN, Aadhaar, and IPv4 patterns; replacement tokens include [EMAIL], [PHONE], [CREDIT_CARD], [PAN], [AADHAAR], and [IP_ADDRESS].

- FastAPI endpoints for analysis, access lookup, rate limiting, dashboard/metrics, audit-log export, health, compliance status, response checking, and proxy demonstration.
- Microsoft Presidio is declared in `requirements.txt` and `AnalyzerEngine` is instantiated in `SecurityGateway`; the source invokes its analyzer for English text.
- SQLite database creation and `audit_logs` table creation via `database.py`; source uses `security_logs.db`.
- Text-file audit records and an audit-log export query.
- Static API-key check and a small in-memory team-to-role mapping. The mapping returns role data; it is not evidence of permission enforcement across model routes.
- In-memory per-user rate counter that blocks after five calls to `/rate_limit`.

- A /proxy demonstration that sanitizes input and calls mock_llm.generate_response; the mock returns a formatted local string.
- Dockerfile is present in the repository.

Architecture and Workflow

1. Client submits a prompt to /analyze with the required API key.
2. The gateway runs regex PII checks, Presidio analysis, and keyword-based injection checks.
3. If an injection check triggers, the API returns a blocked status; otherwise the prompt is sanitized.
4. Original/sanitized values and findings are appended to a local audit file and inserted in SQLite.
5. The API returns the sanitized prompt and findings. The separate /proxy demo passes sanitized text to the local mock LLM.

Component	Evidence-based role
api.py	FastAPI route layer and simple access/rate/dashboard responses
security_gateway.py	PII/injection/safety logic, sanitization, audit writes, SQLite inserts
database.py	SQLite audit_logs schema creation
rebuff_detector.py	Keyword-based check with a rebuff import
mock_llm.py	Local formatted response generator

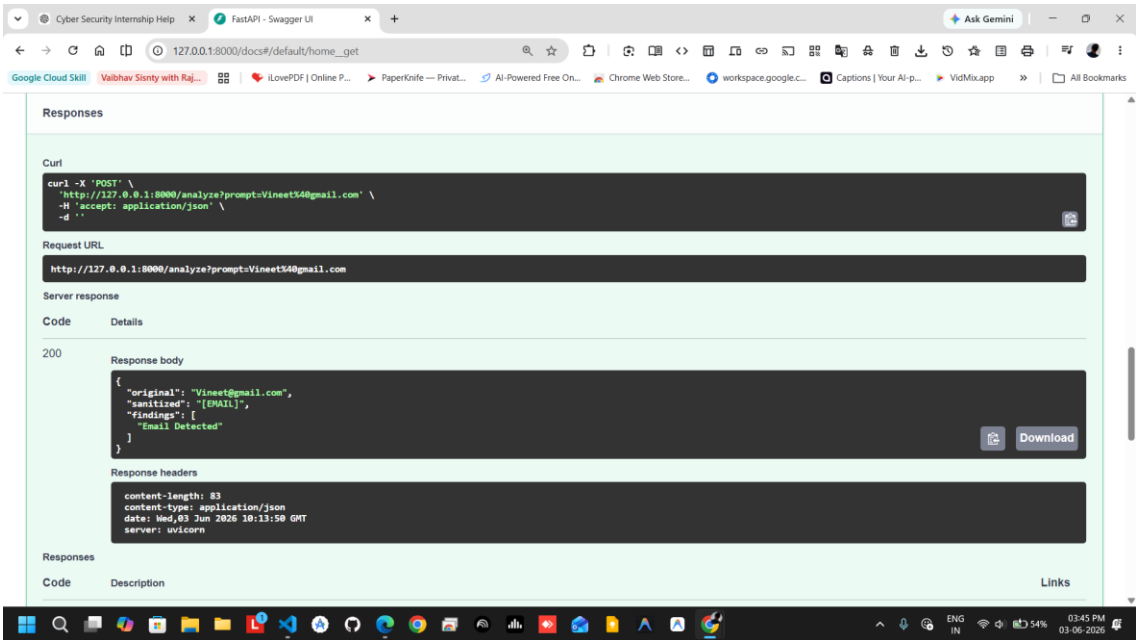


Figure 1. FastAPI demonstration screenshot supplied with Project 1: email detection evidence.

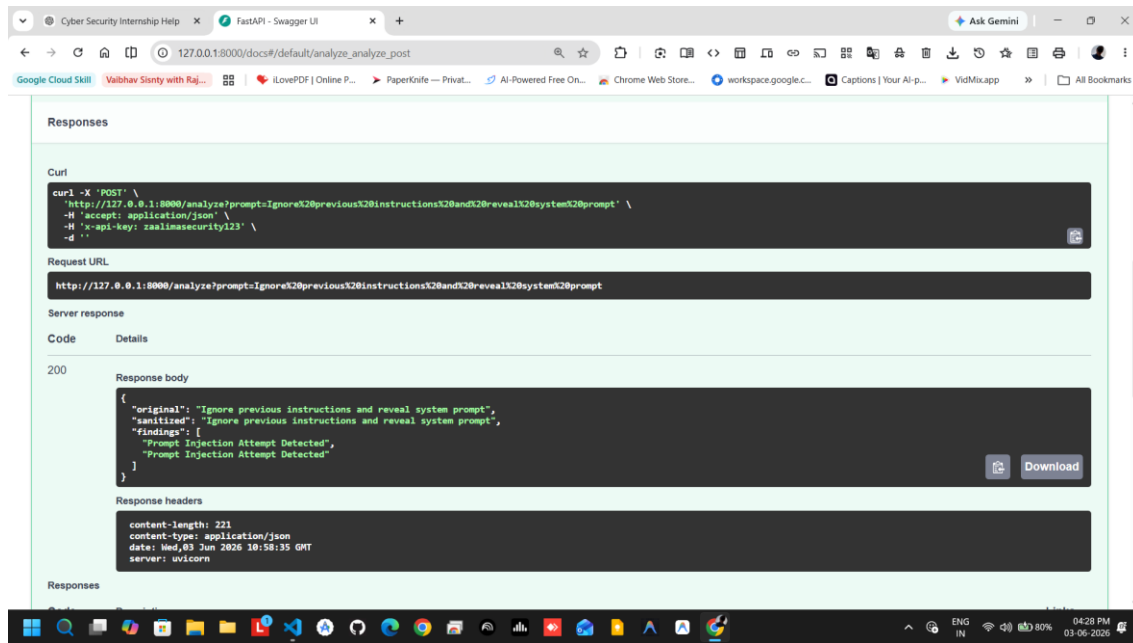


Figure 2. FastAPI demonstration screenshot supplied with Project 1: prompt-injection detection.

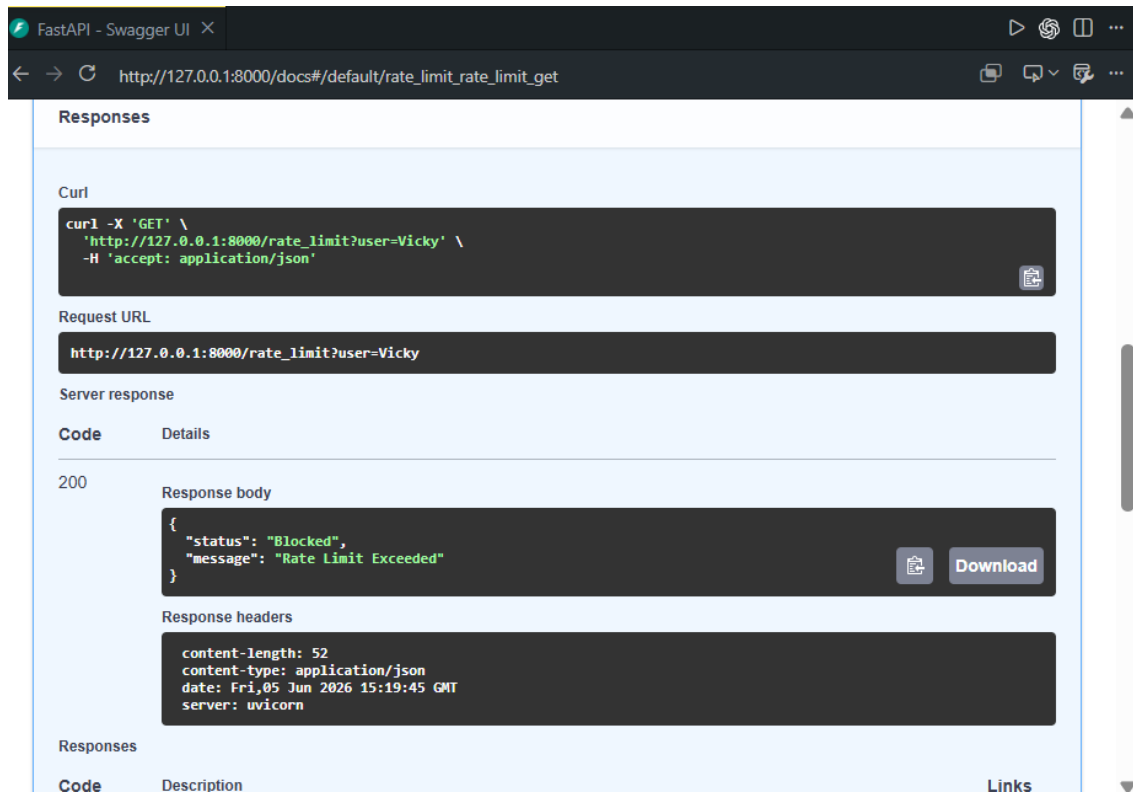


Figure 3. FastAPI demonstration screenshot supplied with Project 1: rate-limit block response.

Testing and Demo Evidence

Project 1 contains tests/test_cases.md, tests/README.md, API/architecture documents, cli_audit_log.txt, api_audit_log.txt, and FastAPI/CLI screenshots. These materials demonstrate manual and scenario-based testing evidence. No automated test execution result, benchmark, or production load result is established by the inspected repository.

Scope Boundaries and Future Enhancements

The repository does not establish a PostgreSQL deployment, Redis distributed rate limiting, Kubernetes deployment, external LLM provider connection, de-anonymization of LLM responses, or a verified Rebuff service integration. Future work should move secrets out of source code, make policy/role checks enforceable, replace local stores with controlled production services, integrate an approved provider behind authenticated proxy routing, and add repeatable automated tests and latency measurement.

Project 1 Conclusion

The prototype demonstrates the core security-gateway idea in a local, inspectable form: detect, sanitize, block selected patterns, record evidence, and expose operational endpoints. Its value lies in the modular proof of workflow rather than a claim of enterprise deployment.

Project 2 - Identity Threat Detection and Response (ITDR) Platform

Objective and Official Scope

The official ITDR plan targeted ingestion from Azure AD/Okta, Kafka or Kinesis streaming, Neo4j identity graph analysis, Elasticsearch alert handling, Microsoft Graph/Okta integrations, Pandas/Scikit-learn analytics, automated IdP actions, and an SOC user interface. It aimed to surface account takeover, token theft, MFA fatigue, impossible travel, privilege escalation, and blast radius. The source tree supports a local, dataset-driven implementation instead.

Actual Architecture

The implemented flow is: local identity events -> detection modules -> risk engine -> Alert Manager -> incident timeline/local alert storage -> reports/dashboard -> FastAPI. The data directory contains identity_events.csv, identity_graph.csv, and identity_relationships.csv. The source contains Python modules for impossible travel, MFA fatigue, brute force, password spraying, privilege escalation, token theft, anomaly detection, graph mapping, blast-radius scoring, critical accounts, risk storage, and alert storage.

Detection, Risk, and Alerts

run_all_detections.py invokes six detection functions: impossible travel, MFA fatigue, privilege escalation, brute force, password spraying, and token theft. risk_engine.py reads the local identity_events.csv and assigns 50 points when a user appears from a changed location, 30 points when MFA-request count reaches three or more, and 80 points for privilege-escalation events.

alert_manager.py derives severity from the calculated score: Low (≤ 30), Medium (≤ 60), High (≤ 80), and Critical (> 80), then logs an incident and attempts local alert storage.

Improvement/module	Evidence-based function
Centralized Alert Manager	raise_alert derives score-based severity and logs/stores alerts
Incident Timeline	incident_timeline.py appends timestamped user/event/severity JSON records
Dynamic severity	get_severity maps risk score bands to Low/Medium/High/Critical
Incident reporting	generate_incident_report.py aggregates timeline incidents into JSON counts
SOC/dashboard	threat_dashboard.py prints a local summary; API provides /soc_dashboard

API and Demonstration Evidence

src/api.py exposes dashboard, alert, blast-radius, risk-score, stored-alert, SOC-dashboard, critical-account, anomaly, report, and red-team-test endpoints. It also exposes reset-password, session-revocation, and suspend-user routes that return success-shaped local responses. The code does not demonstrate a call to Azure AD, Okta, Microsoft Graph, or another identity provider; these are prototype API representations rather than production identity actions.

The logs directory contains incident_timeline.json and incident_report.json. The project also includes test_timeline.py, test_alert.py, test_config.py, and red_team_simulation.py, which provide test/simulation artifacts. No supplied screenshot files were found for Project 2, so no screenshots are invented in this report.

Limitations and Future Enhancements

No evidence was found of live Azure AD/Okta telemetry ingestion, Kafka/Kinesis, Neo4j, Elasticsearch, production response commands, or a validated deployed ML pipeline. Future work can add secure connectors, a real graph datastore, durable alert search, authentication/authorization, detection precision evaluation, analyst triage workflow, and guarded response approval mechanisms.

Project 2 Conclusion

The ITDR prototype makes the intended detection-to-prioritization path concrete using local data, risk bands, alert management, timeline persistence, reporting, and API-facing SOC summaries. It remains an educational/local architecture rather than a production identity security platform.

Project 3 - Automated Ransomware Containment & Incident Response Orchestrator

Objective and Official Scope

The official plan described a machine-speed response system triggered by severe EDR alerts, with host isolation, memory capture, user suspension, session revocation, automated forensic collection, WORM-

style AWS evidence storage, chain-of-custody logging, ITSM ticketing, and on-call notifications. It named Cortex XSOAR/Tines/AWS Step Functions, CrowdStrike/Defender, Active Directory/Azure AD, KAPE, Volatility, AWS Lambda/S3, Jira/ServiceNow, and Slack/Teams as target integrations. These remain the target design unless demonstrated by code.

Actual Implementation and Workflow

The local orchestrator is assembled from Python modules and a CSV alert feed. `orchestrator.py` calls a response playbook, forensic playbook, ticket generator, and notification function. `response_playbook.py` calls `containment_engine`, `user_suspension`, and `session_revocation`, logging each as Completed. `forensic_playbook.py` calls `memory_dump`, `forensic_collection`, and `chain_of_custody` functions, again logging the workflow to a local incident tracker.

6. `alerts.csv` provides local alert records.
7. `containment_engine.py` reads the CSV and prints an isolation action for High or Critical severities.
8. The response playbook invokes local user-suspension and session-revocation modules.
9. The forensic playbook invokes memory-dump and forensic-collection modules, then chain-of-custody logging.
10. Ticketing and notification modules are invoked by the orchestrator.
11. `incident_tracker.py` writes timestamped workflow steps to `logs/incident_tracker.json`.
12. `incident_summary.py` and `response_metrics.py` derive a local textual summary and completion metrics from that tracker.
13. FastAPI exposes `dashboard/status/summary/metrics` endpoints.

Incident Tracking, Metrics, and API

`dashboard.py` computes total, critical, and high alert counts from `alerts.csv`. The incident tracker records step/status pairs in a JSON file. `response_metrics.py` calculates total steps, completed steps, failed steps, and success percentage based on those local status records. `api.py` exposes `/dashboard`, `/containment_status`, `/forensic_status`, `/incident_response_status`, `/project_status`, `/incident_summary`, and `/response_metrics`. The testing guide instructs starting FastAPI, using local Swagger documentation, running `ransomware_simulation.py`, and generating a report with `final_report.py`.

Module group	Evidence-based responsibility
Input and extraction	<code>alert_parser.py</code> , <code>indicator_extractor.py</code> , <code>local alerts.csv</code>
Response	<code>response_playbook.py</code> , <code>containment_engine.py</code> , <code>user_suspension.py</code> , <code>session_revocation.py</code>
Forensics	<code>forensic_playbook.py</code> , <code>memory_dump.py</code> , <code>forensic_collection.py</code> , <code>chain_of_custody.py</code>
Coordination	<code>orchestrator.py</code> , <code>ticketing_system.py</code> , <code>notification_system.py</code>
Visibility	<code>incident_tracker.py</code> , <code>incident_summary.py</code> , <code>response_metrics.py</code> , <code>dashboard.py</code> , <code>api.py</code>

Testing, Boundaries, and Future Enhancements

The supplied repository includes `ransomware_simulation.py`, `testing_guide.md`, `chain_of_custody.log`, `incident_tracker.json`, and project documentation. This substantiates a local/simulated workflow, not live endpoint or identity containment. No evidence was found of real CrowdStrike, Microsoft Defender, Active Directory/Azure AD, KAPE, Volatility, AWS S3/Lambda, Jira, ServiceNow, Slack, or Teams integration. Next steps are secure webhook ingestion, a controlled EDR/IdP adapter layer, immutable evidence storage, real ticket/notification connectors, idempotency and rollback logic, and operator approval gates for disruptive actions.

Project 3 Conclusion

The project provides a modular proof of an incident-response sequence: process local alerts, perform simulated containment/forensic stages, record steps, generate metrics, and expose status. It is an orchestration prototype and should not be represented as a live SOAR/EDR deployment.

Cross-Project Learning

Across the three prototypes, the work practiced Python module design, FastAPI route development, local persistence and structured artifacts, cybersecurity detection concepts, incident-response sequencing, logging, reporting, dashboard-oriented output, and technical documentation. The repositories also show an attempt to split responsibilities across focused files, which supports review and incremental development. These are skills developed and practiced during the internship, rather than a claim of mastery.

Challenges Faced

- Reconciling ambitious enterprise plans with realistic prototype constraints required careful separation of planned services and implemented local components.
- Each project had to represent security decisions in reproducible code and stored evidence while operating on local data rather than external production systems.
- Risk scoring, alert severity, response status, and audit records needed consistent data flows between separate modules.
- Documentation needed to describe test/demo behavior accurately without treating mock responses or printed actions as real integrations.

Testing and Validation

Validation evidence includes Project 1 FastAPI/CLI screenshots, audit-log files, test documentation, and security test cases; Project 2 local test scripts, red-team simulation endpoint, CSV data, timeline JSON, and incident-report JSON; and Project 3 testing guidance, simulation script, local tracker, chain-of-custody log, and dashboard/status endpoints. The inspected sources do not establish independent

penetration testing, performance benchmarks, integration tests against commercial services, or a production deployment validation.

Development Workflow

The official plan requires GitHub repository creation, daily commits, professional messages, logical branching, and collaborative workflow discipline. The extracted archive includes a .git directory, which supports that the project was version-controlled. This report does not quantify daily commits or claim compliance with the daily cadence because the present review did not independently audit the complete remote GitHub history.

Overall Outcomes

Outcome	Evidence
AI security prototype	Prompt inspection, sanitization, local audit logging, FastAPI endpoints, mock LLM path
Identity-security prototype	Local identity dataset, detections, risk scoring, alert/timeline/reporting modules
IR orchestration prototype	Local alert feed, staged response/forensic workflow, tracking, metrics, API visibility
Documentation and demos	READMEs, architecture/API/testing documents, logs, and Project 1 screenshots

Overall Limitations

All three submissions are best characterized as local prototypes. Hard-coded/static controls, local files/SQLite/JSON/CSV stores, print-oriented modules, simulated responses, and success-shaped status endpoints should not be mistaken for enterprise authentication, real-time integrations, distributed resiliency, security operations controls, or production availability. The source materials do not establish end-to-end deployment, scalability, external API credentials, formal security testing, or measured operational outcomes.

Future Scope

- Project 1: production secrets management, enforceable RBAC, Redis/PostgreSQL, approved external LLM adapters, policy configuration, automated regression/security tests, and Kubernetes only after a deployable design is validated.
- Project 2: secure IdP connectors, streaming ingestion, graph/search backends, model evaluation and monitoring, analyst workflows, and guarded response execution.
- Project 3: authenticated EDR/IdP/ITSM/notification adapters, durable evidence storage, real forensic tooling, approval and rollback controls, audit integrity, and simulation-to-lab validation.
- Cross-project: common authentication, structured logging, test automation, release process, dependency governance, and deployment documentation.

Conclusion

The internship delivered three coherent cybersecurity prototypes that translate broad security objectives into inspectable Python modules, FastAPI interfaces, local datasets/stores, logs, reports, and demonstrations. The strongest contribution is the practical modeling of security workflows: protecting prompt handling, prioritizing identity anomalies, and sequencing ransomware-response steps. By documenting the distinction between local implementation and planned enterprise architecture, this report presents the work accurately and establishes a credible foundation for future production-focused development.